

Automated Sim-to-Real Deployment Framework for Reinforcement Learning Control on Resource-Constrained Microcontrollers

Nidharshan V.
Dept. of Computer Science & Eng.
University of Moratuwa
Katubedda, Sri Lanka
nidharshan.23@cse.mrt.ac.lk

Yasantha Niroshana
RoboticGen (Pvt) Ltd
Mount Lavinia, Sri Lanka
yasantha@roboticgen.co

Odil Janandith
RoboticGen (Pvt) Ltd
Mount Lavinia, Sri Lanka
odil@roboticgen.co

Sulochana Sooriyaarachchi
Dept. of Computer Science & Eng.
University of Moratuwa
Katubedda, Sri Lanka
sulochanas@cse.mrt.ac.lk

Keywords—Sim-to-Real Transfer, Reinforcement Learning, Microcontroller Deployment, TinyML, Dynamic Self-Righting.

I. INTRODUCTION

Deep reinforcement learning (RL) enables agile motor control in robotics [1], [2]. While transferring physics-trained controllers to compute-heavy robots with Linux and ROS 2 is well established [3], deploying policies to resource-constrained microcontrollers (MCUs) such as the ESP32 (<520 kB RAM, no OS) remains entirely manual. Developers must hand-code network weights into C arrays, implement forward inference kernels, and rewrite peripheral drivers whenever model architectures or hardware pins change.

To address this gap, we present *simtoreal*, an open-source Python library that automates policy deployment to microcontrollers. Our contributions are: (1) a dual-target code exporter converting trained Stable-Baselines3 or PyTorch actors into zero-allocation C headers or precompiled MicroPython bytecode (.mpy); (2) a hardware abstraction synthesizer generating the complete on-board control loop (main.py) from a declarative YAML specification; and (3) empirical benchmarking of numerical equivalence, embedded resource overhead on an ESP32, and dynamic self-righting on a 2-DOF bipedal robot (*RoboI*).

II. LITERATURE REVIEW

Sim-to-Real Transfer: The discrepancy between simulation and the real world (the reality gap) is primarily addressed via domain randomization (DR) [4] and actuator system identification [1], [5]. While modern simulators enable large-scale DR training [2], [3], standard deployment runtimes (ONNX, TorchScript) require tens of megabytes of RAM, far exceeding MCU limits.

Embedded Machine Learning (TinyML): Frameworks such as TensorFlow Lite for Microcontrollers (TFLM) [6] and MCUNet [7] support MCU inference, but focus on vision and audio tasks with general tensor interpreters that add needless overhead for compact control MLPs. RLtools [8] enables embedded C++ execution, but requires training within its

custom C++ environment, hindering direct reuse of standard Python RL ecosystems (Gymnasium, Stable-Baselines3).

Benchmarking Embedded Sim-to-Real: Zhou et al. [9] demonstrated that low-cost edge platforms suffer substantial performance degradation without grounded benchmarks. We therefore evaluate numerical parity, closed-loop simulation consistency, and embedded microcontroller resource limits.

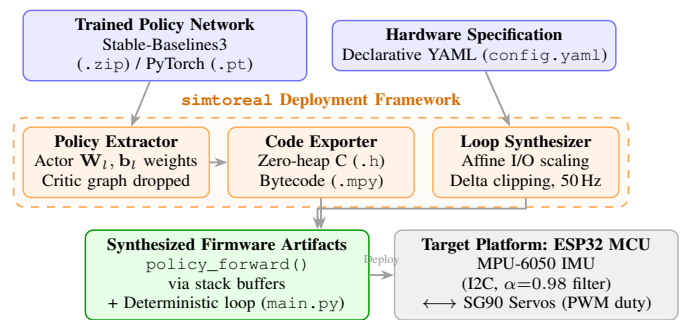


Fig. 1. Detailed architectural workflow of the *simtoreal* deployment framework.

III. MATERIALS AND METHODS

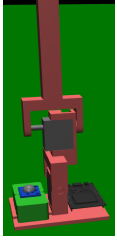
The *simtoreal* pipeline (Fig. 1) automates the translation of simulation-trained policies into functional embedded firmware through three main stages.

1) Policy Extraction: The framework parses trained Stable-Baselines3 PPO models (.zip) or PyTorch checkpoints (.pt), extracting layer weights W_l , biases b_l , and activations (tanh, ReLU). Critic networks and optimizer states are discarded, isolating the forward actor graph.

2) Dual-Target Code Generation: Network parameters can be exported via two distinct backends: *a) Zero-Heap C Header:* Parameters are exported as compile-time const float arrays stored in Flash. Forward inference runs in `policy_forward()` via two statically allocated stack buffers that alternate ping-pong style. Dynamic heap allocation (malloc) is entirely avoided, preventing runtime memory fragmentation. *b) Precompiled MicroPython Bytecode:* To support rapid prototyping, policies can be exported as MicroPython modules and precompiled to .mpy bytecode via

mpy-cross, eliminating on-device memory exhaustion during source parsing.

3) Control Loop Synthesis: Peripherals and execution parameters are declared in `config.yaml`. The generator writes `main.py`, configuring sensor drivers (MPU-6050 over I2C), affine observation normalization ($\mathbf{o}_t = \mathbf{s} \odot \mathbf{x} + \mathbf{b}$), actuator mapping (delta or absolute modes with angle limits), and an accurate 50 Hz periodic control loop.



(a) MuJoCo simulation



(b) Physical *Robo1* robot

Fig. 2. *Robo1* self-righting bipedal testbed: (a) CAD digital twin in MuJoCo physics simulation, and (b) physical 3D-printed robot with ESP32 and MPU-6050.

4) Experimental Testbed (Robo1): We evaluated the framework on *Robo1* (Fig. 2), a 2-DOF self-righting robot (total mass 94.8 g) with an ESP32 MCU, an MPU-6050 IMU, and two SG90 micro-servos. A matching digital twin was built in MuJoCo using exact CAD masses, spatial inertia tensors, joint limits ($\pm 90^\circ$), damping (0.02 N·m·s/rad), and collision proxies. The observation vector is $\mathbf{o}_t = [\phi_{\text{roll}}, \theta_{\text{pitch}}, a_z, q_1, q_2]$, where a_z disambiguates upright from inverted orientations, and q_1, q_2 represent current joint angles. Actions apply incremental target adjustments ($\Delta q \leq 0.08$ rad) at 50 Hz. Training incorporates domain randomization over mass ($\pm 15\%$), damping/friction ($\pm 20\%$), actuator gain ($\pm 15\%$), and sensor noise ($\sigma = 0.015$ rad).

IV. RESULTS AND DISCUSSION

The framework was benchmarked across numerical precision, simulation fidelity, and microcontroller resource overhead (Table I). The evaluated policy is a 5–64–64–2 tanh MLP (4,674 parameters, 18.7 kB as float32).

Numerical Equivalence and Simulation Fidelity: Across 4,609 host test inputs (2,000 random vectors and 2,609 state samples from simulation), maximum absolute difference against PyTorch was 1.2×10^{-6} for MicroPython and 2.9×10^{-6} for C, confirming single-precision floating-point agreement. Action sign and magnitude agreement was 100%.

In closed-loop MuJoCo simulation over 1,000 episodes across five initial fallen poses, the synthesized C controller produced righting trajectories identical to PyTorch. Across nominal and domain-randomized physics, the policy achieved a 97.2% overall get-up success rate (100% from standard fallen poses in 1.15 s mean; 88.0% from severe inverted poses). Under randomized dynamics, success remained at 97.2% with a modest 20% increase in righting time (1.38 s), confirming high dynamic robustness.

Microcontroller Resource and Latency Profile: On an ESP32 running MicroPython 1.28, the precompiled .mpy policy matched PyTorch outputs within 9.5×10^{-7} . The compiled binary occupies 75.1 kB in Flash. In RAM, MicroPython object

TABLE I
EMPIRICAL VERIFICATION AND BENCHMARK METRICS

Benchmark / Test	Target / Source	Measured Result
Host Parity ($n=4,609$)	MicroPython	Max err: 1.2×10^{-6}
Host Parity ($n=4,609$)	C Header	Max err: 2.9×10^{-6}
Simulation (1,000 runs)	C Header	97.2% success (= PyTorch)
ESP32 Flash Footprint	Flash Storage	75.1 kB
ESP32 Dynamic RAM	MicroPython Heap	119.6 kB (72% of heap)
Inference Latency	MicroPython	155.2 ± 7.4 ms ($7.8 \times$ budget)
Sensor Noise (still)	MPU-6050	1.6 mrad (sim: 15 mrad)
Hardware I/O Time	I2C IMU + PWM	4.2 ms (21% of budget)
Loop Rate	MicroPython Loop	5.4 Hz (inference bound)

representations allocate 119.6 kB of dynamic memory (72% of the 162.3 kB free heap).

On-device timing indicates that MicroPython inference requires 155.2 ± 7.4 ms per decision. In bench tests with sensors and servos connected, reading the MPU-6050 over I2C takes 3.2 ms and servo PWM updates require 0.95 ms. Total hardware I/O requires only 4.2 ms, easily fitting within the 20 ms (50 Hz) real-time budget. However, because MicroPython software inference takes ~ 182 ms under full loop memory management, the complete interpreted loop runs at 5.4 Hz. Static sensor noise was measured at 1.3–1.6 mrad (0.2 mrad with complementary filtering), well within the 15 mrad noise injected during training. These benchmarks show that while MicroPython is effective for rapid prototyping, compiled C header deployment is essential for achieving real-time 50 Hz control.

V. CONCLUSION

The `simtoreal` framework bridges standard Python reinforcement learning tools and resource-constrained microcontroller deployment. By combining automated policy extraction, zero-heap C and MicroPython code generation, and declarative control loop synthesis, it eliminates manual firmware translation. Empirical evaluations confirm numerical parity and 97.2% simulation get-up success on the *Robo1* bipedal testbed. Microcontroller profiling demonstrates that peripheral I/O easily satisfies 50 Hz deadlines, while highlighting software-interpreted inference as the key performance bottleneck. Future work entails deploying the compiled C backend onto the physical robot to conduct real-world dynamic self-righting trials.

REFERENCES

- [1] J. Hwangbo *et al.*, “Learning agile and dynamic motor skills for legged robots,” *Sci. Robot.*, vol. 4, no. 26, p. eaa5872, 2019.
- [2] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep RL,” in *Proc. CoRL*, PMLR vol. 164, 2022, pp. 91–100.
- [3] M. Mittal *et al.*, “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE RA-L*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [4] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *Proc. IEEE/RSJ IROS*, 2017, pp. 23–30.
- [5] J. Tan *et al.*, “Sim-to-real: Learning agile locomotion for quadruped robots,” in *Proc. RSS*, 2018.
- [6] R. David *et al.*, “TensorFlow Lite Micro: Embedded ML on TinyML systems,” in *Proc. MLSys*, vol. 3, 2021, pp. 800–811.
- [7] J. Lin *et al.*, “MCUNet: Tiny deep learning on IoT devices,” in *Proc. NeurIPS*, 2020, pp. 11711–11722.
- [8] J. Eschmann, D. Albani, and G. Loianno, “RLtools: A fast, portable deep RL library for continuous control,” *J. Mach. Learn. Res.*, vol. 25, pp. 1–19, 2024.
- [9] R. Zhou, O. Tavaillae, S. Chen, and A. Y. Zomaya, “Measure the sim-to-real gap: An affordable benchmark platform for RL in AIoT systems,” arXiv:2607.10309, 2026.